

SIMATS ENGINEERING
Assessment Tool 3 - Descriptive Written Test

Course: Artificial Intelligence | Course Code: CSA1707

Duration: 45 Minutes | Total Marks: 20

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.
(BL3)

Name: Eshwanth C | Reg No: 192321062

1. Intelligent Agents in a Smart Home Automation System

Question: A smart home automation system manages lighting, temperature, and security based on user behavior and environmental conditions. It must respond to real-time inputs (motion, temperature, time of day) while learning user preferences. Explain the simple reflex, model-based, goal-based, and utility-based agents, analyze how each would function in this system, and justify the most effective type or hybrid approach for comfort, energy efficiency, and security.

A smart home automation system receives inputs such as motion, temperature, time of day, and user preferences. An intelligent agent observes the environment through sensors and takes suitable actions through devices such as lights, fans, air conditioners, alarms, and locks. The four important types of agents are the Simple Reflex Agent, Model-Based Agent, Goal-Based Agent, and Utility-Based Agent.

1. Simple Reflex Agent

A simple reflex agent selects an action based only on the current percept, using condition-action rules.

Example:

- If motion is detected → turn ON the light.
- If temperature is above 28°C → turn ON the fan.
- If no motion is detected → turn OFF the light.

Working in smart home: The agent continuously receives sensor information and immediately responds according to predefined rules.

Advantages: Simple to design; fast response; easy to implement.

Disadvantages: Does not remember previous situations; cannot learn user preferences; cannot handle complex situations effectively.

2. Model-Based Agent

A model-based agent maintains an internal model/state of the environment. It remembers previous information and uses it along with the current sensor input.

Example: The system knows that the user normally returns home at 6 PM, the living-room light was previously switched ON, and the current temperature is 30°C. The agent uses this to decide whether to turn on the lights or adjust the temperature.

Working in smart home: It maintains the state of rooms, devices, users, temperature, and security conditions.

Advantages: Maintains previous information; works better in partially observable environments; makes better decisions than a simple reflex agent.

Disadvantages: More complex; requires memory and an internal model.

3. Goal-Based Agent

A goal-based agent chooses actions that help it achieve a particular goal. Possible goals in a smart home: maintain comfortable temperature, reduce electricity consumption, keep the house secure, and provide suitable lighting.

Example - Goal: Save energy. The agent may:

Detect that nobody is in a room.

Check that the lights are ON.

Turn OFF unnecessary lights.

Keep essential security devices active.

The agent selects actions based on whether they help achieve the desired goal.

Advantages: Can plan actions; handles different situations; more flexible than reflex agents.

Disadvantages: Requires more computation; selecting the best action can take more time.

4. Utility-Based Agent

A utility-based agent not only tries to achieve a goal but also chooses the action that gives the highest utility or overall benefit. In a smart home, utility can consider comfort, energy consumption, security, cost, and user preferences.

Example: At 30°C the agent could turn the AC ON at high power, turn the fan ON, or set the AC to moderate cooling. It compares the choices and selects the one giving the best balance between comfort and energy efficiency.

Comparison

Agent Type	Main Idea	Smart Home Example
Simple Reflex	Current condition → action	Motion detected → light ON
Model-Based	Uses current and previous state	Remembers room/device status
Goal-Based	Works toward a goal	Maintain comfortable temperature
Utility-Based	Chooses the best overall option	Comfort + low energy + security

Most Effective Approach

A hybrid approach using model-based, goal-based, and utility-based agents would be most effective. The model-based component maintains information about the home and user behavior. The goal-based component defines objectives such as comfort and security. The utility-based component selects the best action by balancing comfort, energy consumption, cost, and security.

For example, when the user enters the house, the system recognizes the user's normal behavior, turns on required lights, adjusts the temperature, and avoids unnecessary energy consumption.

Conclusion

A simple reflex agent is useful for basic immediate actions but cannot handle complex smart-home requirements. A model-based agent provides memory, a goal-based agent provides planning, and a utility-based agent helps select the best action. Therefore, a hybrid intelligent-agent system provides the best solution for comfort, energy efficiency, and security.

2. UCS and IDS for a Robot in an Unknown Maze

Question: A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss their advantages and disadvantages in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

A robot in an unknown maze needs to find a path from its starting position to the exit. Since it has no prior knowledge of the maze, it can use uninformed search algorithms. Two suitable strategies are Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).

Uniform Cost Search (UCS)

UCS expands the node having the lowest path cost from the starting point. It uses a priority queue.

Working (start position S, exit G):

Start from the initial position.

Add the starting node to the priority queue.

Select the node with the lowest path cost.

Expand that node.

Add its unvisited neighboring positions to the queue.

Again select the lowest-cost node.

Continue until the goal is reached.

If every movement has the same cost, UCS behaves similarly to Breadth-First Search (BFS).

Example: With three paths - A (cost 10), B (cost 6), C (cost 8) - UCS selects Path B first because its current cost is the lowest.

Advantages:

- Complete when step costs are positive.
- Finds the least-cost path.
- Useful when different movements have different costs.
- Does not require prior knowledge of the maze.

Disadvantages:

- Can consume a large amount of memory.
- Can be slower when there are many paths.
- Stores many frontier nodes.

Complexity (b = branching factor, C^* = optimal solution cost, ϵ = minimum step cost):

Time: $O(b^{(1 + C^*/\epsilon)})$ | Space: $O(b^{(1 + C^*/\epsilon)})$

Thus, UCS can require significant time and memory for a large maze.

Iterative Deepening Search (IDS)

IDS combines the advantages of Depth-First Search (DFS) and Breadth-First Search (BFS). It repeatedly performs depth-limited DFS with increasing depth limits (0, 1, 2, 3, ...) until the robot finds the exit.

Example: If the exit is at depth 4: Limit 0 → Start; Limit 1 → search up to level 1; Limit 2 → up to level 2; Limit 3 → up to level 3; Limit 4 → find Exit.

Advantages:

- Uses much less memory than BFS.

- Complete when the branching factor is finite.
- Finds the shallowest solution when all step costs are equal.
- Suitable when the solution depth is unknown.

Disadvantages:

- Repeats the search from the beginning at each depth, causing additional computation.
- Not suitable for different path costs because it considers depth rather than actual cost.

Complexity (b = branching factor, d = depth of shallowest solution):

Time: $O(b^d)$ | Space: $O(bd)$

Therefore, IDS has significantly lower memory requirements than UCS in many cases.

UCS vs IDS Comparison

Feature	UCS	IDS
Search basis	Path cost	Depth
Memory	High	Low
Best use	Different movement costs	Unknown solution depth
Main drawback	High memory usage	Repeated searching

Relation to Intelligent Agents

The robot is an intelligent agent because it observes its environment using sensors, receives information about walls and available paths, selects actions based on the available information, moves toward the goal, and updates its knowledge as it explores the maze.

Simple Reflex Agent: Reacts to immediate conditions (e.g., if wall ahead → turn right). It may fail to find the best path because it does not remember previous states.

Model-Based Agent: Maintains an internal representation of the maze - visited locations, blocked paths, available paths, and current position. This is more suitable for an unknown maze.

Goal-Based Agent: Focuses on reaching the exit and evaluates movements by whether they help reach the goal.

Utility-Based Agent: Chooses a path based on factors such as shortest distance, lowest movement cost, safety, and battery consumption. Useful when different paths have different costs.

Best Combination for the Maze

For an unknown maze, a model-based or goal-based agent combined with an appropriate uninformed search algorithm is effective.

If all movements have equal cost: Model-Based Agent + IDS – IDS requires less memory, can find a shallow solution, needs no prior knowledge of the maze, and works well when the exit depth is unknown.

If movement costs differ: Model-Based Agent + UCS – UCS considers actual path cost and finds the least-cost path.

Final Recommendation

For a normal maze where every movement has the same cost, Model-Based Agent + IDS is a suitable combination. The model-based agent remembers the explored maze, while IDS searches progressively deeper until it finds the exit. If different paths have different costs, UCS should be preferred because it guarantees the least-cost solution under its usual positive-cost assumptions.